ReTreever: Hierarchical Retrieval at Scale

Bridging Interpretability & Efficiency

Shubham Gupta[1,3,4] **Zichao Li**[1,2,4] **Tianyi Chen**[1] **Cem Subakan**[2,4] **Siva Reddy**[1,2,4] **Perouz Taslakian**[1,2,4] **Valentina Zantedeschi**[1,3]

1 ServiceNow Research 2 McGill University 3 Université Laval 4 Mila – Québec AI Institute

Hierarchical retrieval that scales

Dense retrieval is fast but a black box. Hierarchical methods organize data into an inspectable structure.

✓ What hierarchy gives you

Data organization
Nodes of progressively finer semantic groupings, preserving structure (document sections, entity relationships).

Interpretability
Probe nodes to see what they contain and why they lead to certain content (correct or incorrect).

✕ But does not scale

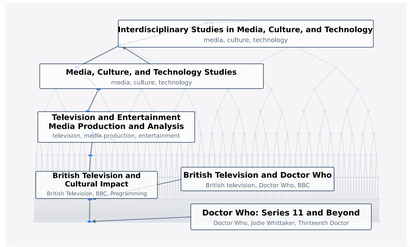
LLM-bound
Prompts an LLM at every node of the hierarchy.

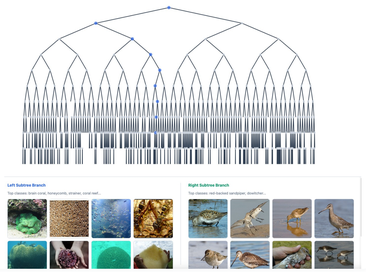
Slower than dense retrieval
RAPTOR takes ~1.7 s/query on NQ.

*Sarthi, et al ICLR 2024 — **Raptor: Recursive abstractive processing for tree-organized retrieval***

RETREEVER builds and navigates the tree entirely on embeddings — no LLM calls.

Semantic groupings emerge automatically





Method: a tree learned for retrieval

Learn a binary tree that organizes embeddings into semantic hierarchies, trained end-to-end for retrieval.

1 Encode

Pretrained encoder → embeddings
Query / Document

2 Route

Binary tree sorts left / right at each node

3 Match & inspect

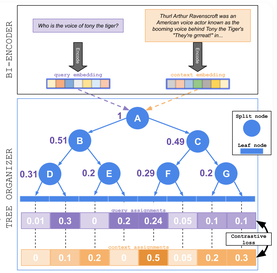
Leaf score distributions match

Path through tree = explanation

Every routing decision is auditable — the tree path explains how documents were grouped.

Architecture: encoder + differentiable tree organizer

- **Bi-encoder**
  Query and document are embedded independently into a shared space.
- **Soft routing**
  Each split node emits a distribution over its children — fully differentiable.
- **In practice**
  Node outputs a probability of the text belonging to that node, Probability multiply along the path to the leaf (fully differentiable), Contrastive loss pulls positive (query, reference) pairs to the same leaf
- **Contrastive objective**
  Aligns query and reference leaf assignments, trained end-to-end.



1 Operates on embeddings

No LLM calls : scales to 100M+ docs

2 Coarse-to-fine reps

Index at any tree level; trade speed for accuracy

3 Contrastive training

Pulls positive (query, reference) pairs to the same leaf

Explore more

Scan for the interactive demo, code, and full paper.

Interactive demo **Code** **Paper**

Results

160×	+20.7	9.9 ms	0	100%
faster than Raptor	avg nDCG@10 vs Raptor	avg query latency	LLM calls	inspectable routing

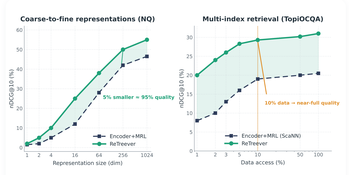
Flexible Search Options

Single-Index Search

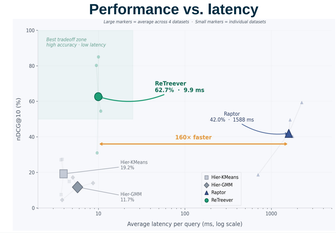
Build one index over all docs at a chosen level; Q scans the full index for the best match.

Multi-Index Search

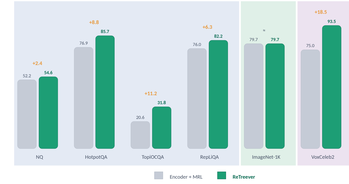
Q routes to its top-k leaves; only those leaf indexes are searched — the rest are skipped (orange = searched).



Performance vs. latency



ReTreever vs. Dense Encoder

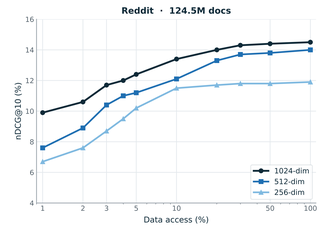


Scales to 124.5M documents

On Reddit Title–Body (~300× bigger than NQ): the learned tree is a memory-friendly multi-index alternative to global ANN.

Built fast, fits in memory
ScaNN ran out of memory (>1 TB RAM).
ReTreever builds the index in 14 min at 1024-dim, 2 min at 256-dim.

Tree stays well-used
Documents spread evenly across leaves 1024 leaves, Routing entropy 8.4 (max 10)



Key Takeaways

1. **Transparent organization**
   Semantic groupings, no clustering objective
2. **Inspectable retrieval**
   Every routing decision is auditable
3. **Coarse-to-fine reps**
   One tree = multiple granularity
4. **Flexible search**
   Single- or multi-index over any level
5. **No tradeoff**
   Top accuracy at lowest latency
6. **No LLM calls**
   Pure embedding navigation